



Container Technology



Lernziele

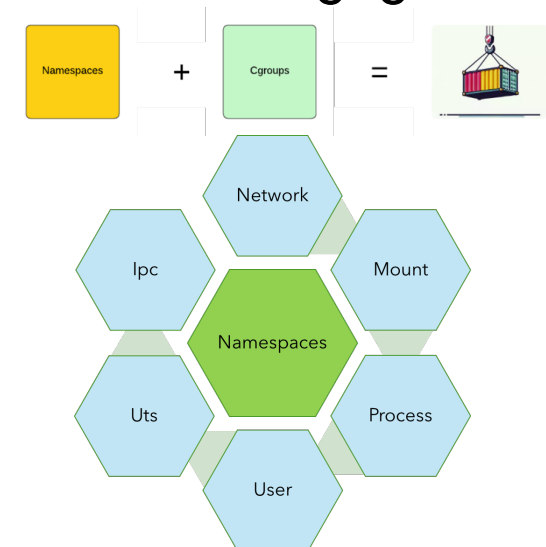
- Sie haben einen Einblick in die Technologie hinter den Containern.

Zeitlicher Ablauf

- Auf welcher Technologie basieren Container?
- Linux Namespaces
- Übung
- Windows?
- Reflexion
- Lernzielkontrolle

Auf welcher Technologie basieren Container?

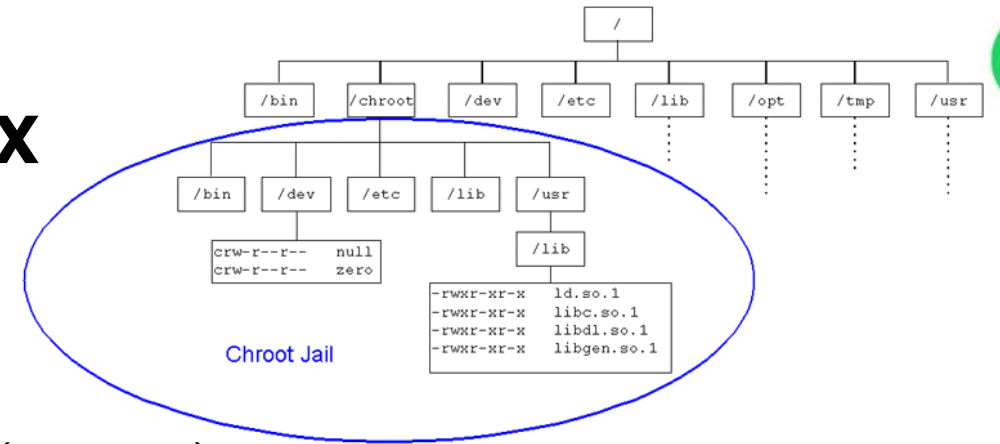
- **Linux Namespaces** sind eine Technologie, die es ermöglicht, verschiedene Systemressourcen wie Prozesse, Netzwerkinterfaces oder Dateisysteme voneinander zu **isolieren**.
- Dies schafft die **Grundlage für Containerisierung**, bei den mehrere **isolierten Umgebungen** auf einem **einzigem Host-System laufen** können, ohne sich gegenseitig zu beeinflussen.
- Verschiedene Linux Namespaces (nicht abschliessend)
 - ◇ Prozess Namespace: Isoliert die **Prozess-IDs**
 - ◇ Network Namespace: Trennt die **Netzwerk** Stacks, inklusive Interfaces, Routing-Tabellen und Ports
 - ◇ Mount Namespace: Isoliert die **Dateisysteme**
 - ◇ Control Groups (**CGroups**): verwaltet und limitiert die Ressourcenverwendung (**Memory, CPU**) von Prozessen



Auf welcher Technologie basieren Container?

- Weitere Informationen
 - ◇ <https://medium.com/@shaibs3/understanding-container-fundamentals-from-concepts-to-practical-demos-79cf6830febf>
 - ◇ <https://docs.docker.com/engine/security/seccomp/>
 - ◇ <https://kubernetes.io/blog/2021/08/25/seccomp-default/> - Kubernetes 1.22
 - ◇ <https://medium.com/faun/the-missing-introduction-to-containerization-de1fbb73efc5>

Container basieren auf Linux Konzepten (vereinfacht)



- **Dateisystem (ls -l /, sudo df -h)**
 - ◇ Alles ist eine Datei z.B. /proc/cpuinfo, /dev/sdaX (Harddisk)
 - ◇ Es gibt keine Laufwerke alles ist via / (root) erreichbar. Mehrere Roots sind möglich.
 - ◇ Die Unterstützung mehrere Filesysteme (ext4, overlay2, aufs etc.) ist die Regel und nicht die Ausnahme.

- **Prozesse (pstree -n -p)**
 - ◇ Prozesse sind hierarchisch angeordnet
 - ◇ Der erste Prozess hat die ID 1, es sind mehrere Hierarchien möglich.

```
systemd(1) +-accounts-daemon(638)
            +-containerd(3867) +-containerd-shim(7978) ---pause(8076)
                                +-containerd-shim(15970) ---metrics-sidecar(15987)
                                +-containerd-shim(16118) ---metrics-server(16133)
                                +-containerd-shim(747040) ---sh(747058) +-nginx(747102) ---nginx(747108)
                                                                    +-php-fpm(747105) +-php-fpm(747106)
                                                                    +-php-fpm(747107)
```

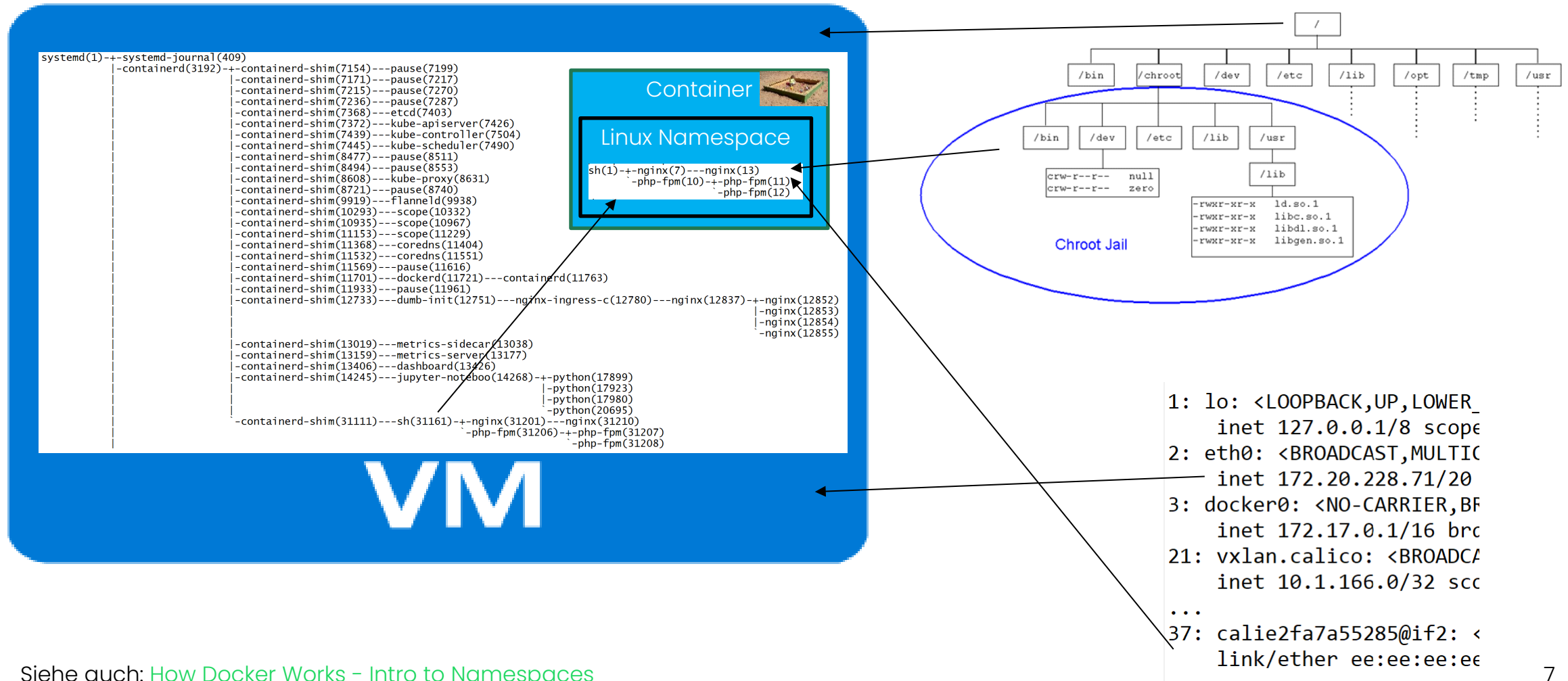
- Es sind **Subnetzwerke** möglich (ip addr)

```
1: lo: <LOOPBACK,UP,LOWER_
   inet 127.0.0.1/8 scope
2: eth0: <BROADCAST,MULTI
   inet 172.20.228.71/20
3: docker0: <NO-CARRIER,BF
   inet 172.17.0.1/16 brd
21: vxlan.calico: <BROADCA
   inet 10.1.166.0/32 scc
```

```
...
37: calie2fa7a55285@if2: <
   link/ether ee:ee:ee:ee
```

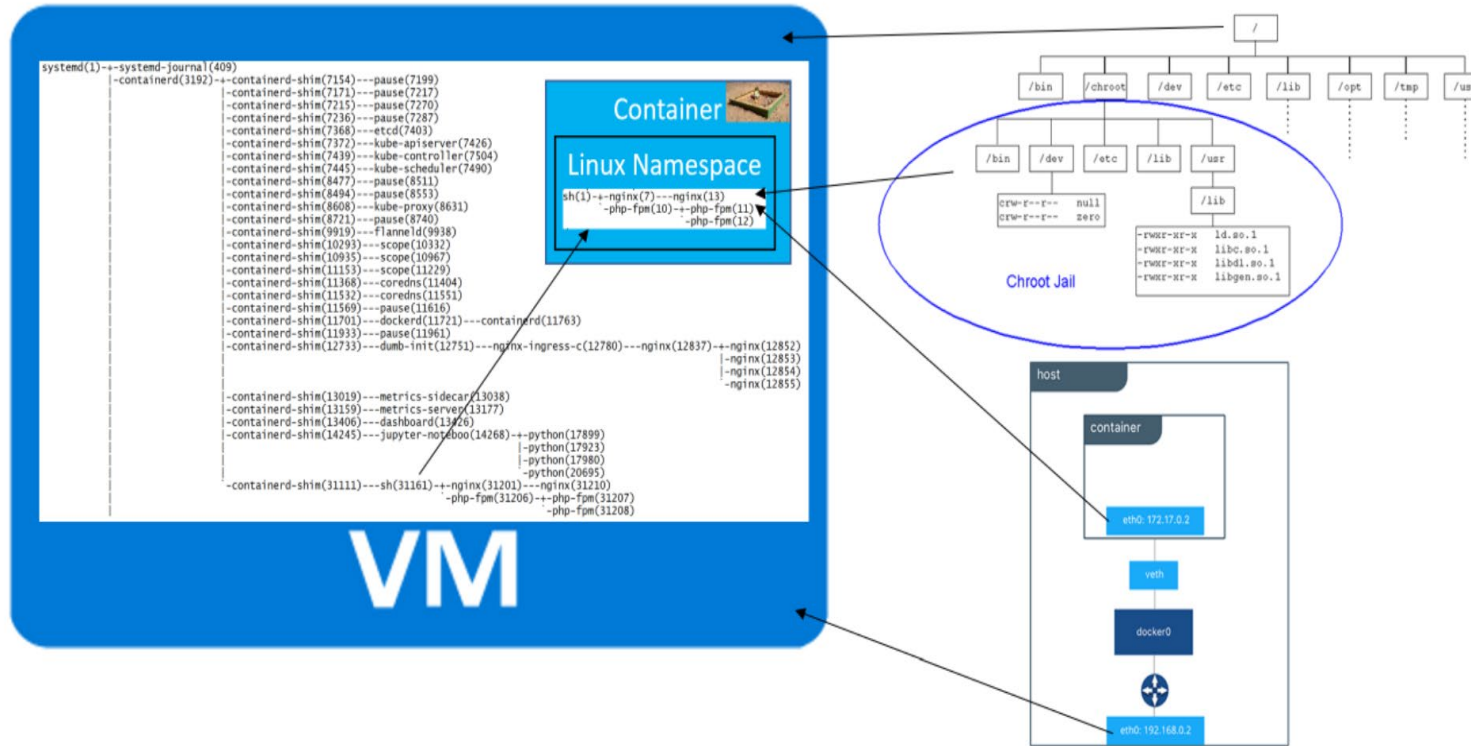
```
sh(1) +-nginx(7) ---nginx(13)
        +-php-fpm(10) +-php-fpm(11)
                    +-php-fpm(12)
```

Virtuelle Maschine vs. Container/Namespaces





Übung 02-1: Linux Namespaces – nsenter

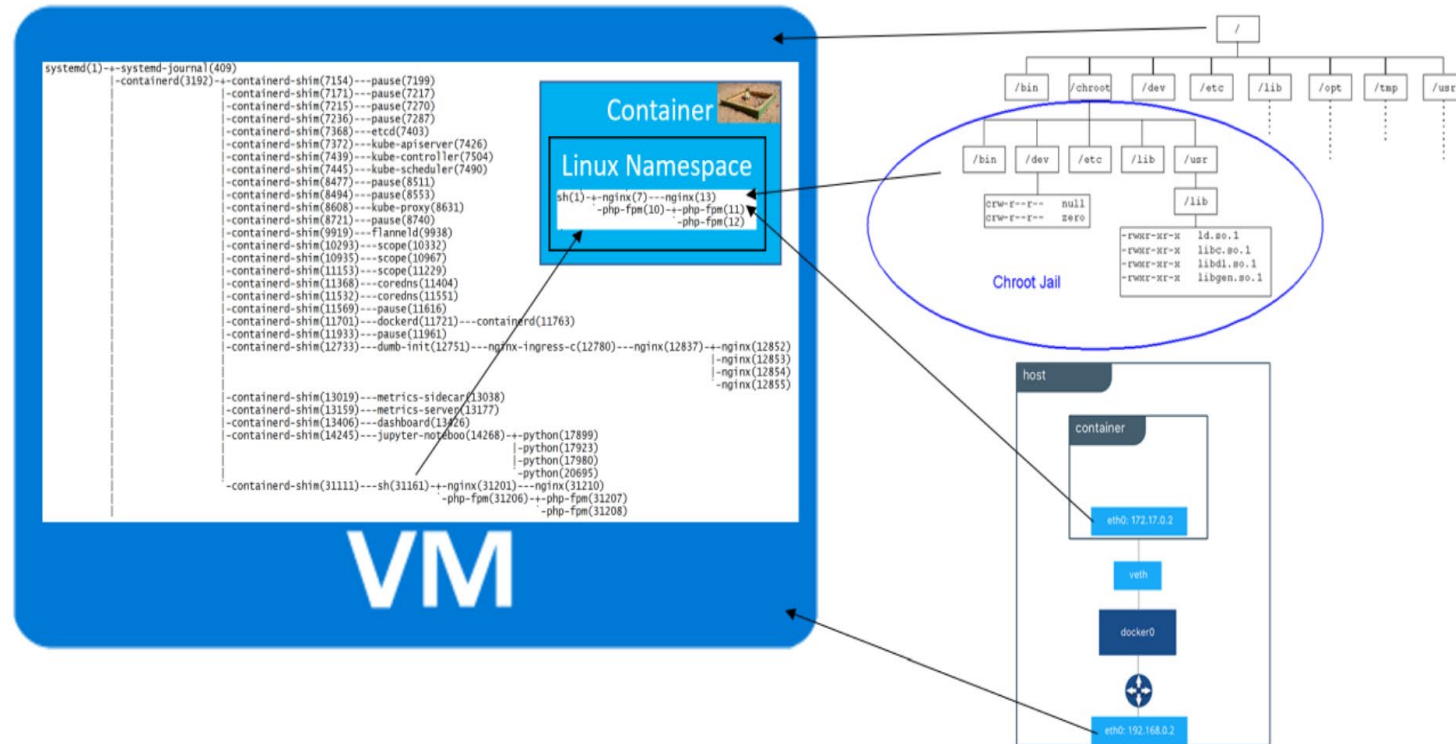


- **Ergebnis:** Container nutzen Linux Namespaces zur Isolation von Prozessen, Netzwerk und Dateisystem. Mit **nsenter** kann man gezielt in diese Namespaces eintreten und Container direkt untersuchen.
- **Erkenntnisse:** Container basieren auf **Kernel-Funktionen**, keine eigene Virtualisierungsschicht. **nsenter** macht sichtbar: Container und Host teilen denselben Kernel – Isolation ist logisch, nicht physisch.

Linux Namespaces sind eine Technologie, die es ermöglicht, verschiedene Systemressourcen wie Prozesse, Netzwerkinterfaces oder Dateisysteme voneinander zu isolieren.



Übung 02-3: Linux Namespaces – unshare

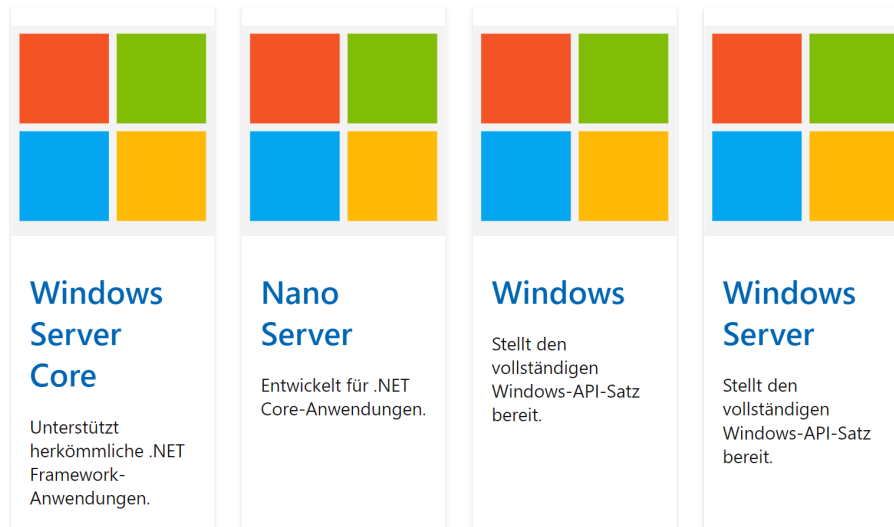


- **Ergebnis:** Container nutzen Linux Namespaces zur Isolation von Systemressourcen. Mit `unshare` lassen sich neue Namespaces manuell starten – quasi ein Container „von Hand“.
- **Erkenntnisse:** Container-Technologien bauen direkt auf Namespace-Mechanismen des Linux-Kernels auf. `unshare` zeigt, wie Container entstehen – `nsenter`, wie man hineingelangt. Damit wird sichtbar: Container sind leichtgewichtige Prozess-Isolation, keine eigenständigen Systeme.

Linux Namespaces sind eine Technologie, die es ermöglicht, verschiedene Systemressourcen wie Prozesse, Netzwerkinterfaces oder Dateisysteme voneinander zu isolieren.

Was ist mit Windows? Windows Container?

Windows Base Images von Microsoft



Der Windows Server Core ist 5GB gross

Der Nano Server 300 MB

...

- Ausführen von Windows- oder Linux-basierte Container unter **Windows 10 für Entwicklung** und Tests mit Docker Desktop.
- Entwickeln, Testen, Veröffentlichen und Bereitstellen von Windows-basierten Containern mithilfe der leistungsfähigen Containerunterstützung in Visual Studio und Visual Studio Code.
- Container Images in **Azure** oder anderen **Public Clouds bereitstellen**.
- Container **lokal** mithilfe von Azure Kubernetes Service in Azure Stack HCI, Azure Stack mit der AKS-Engine oder Azure Stack mit OpenShift bereitstellen.



Reflexion

- Container sind ein altes Konzept
- Sie basieren "vereinfacht" auf Prozessen, die in unterschiedlichen Linux Namespaces laufen

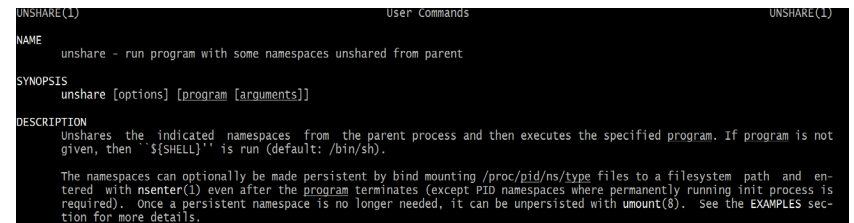
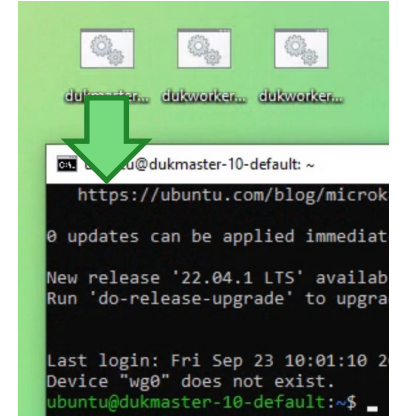
Lernzielkontrolle

- Sie haben einen Einblick in die Technologie hinter den Containern.



Übung: Linux Namespace wechseln (unshare)

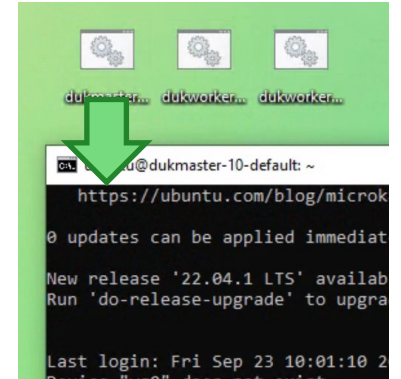
- Wechselt, via Desktop Icon, in die "dukmaster" VM
- Anzeige Namespaces, aktueller Prozess und Anzeige alle Prozesse
 - ◇ `lsns` # Anzeige der Arten von Linux Namespaces
 - ◇ `ps tree -n -p -A -T` # zeigt alle Prozesse hierarchisch (in VM)
- Wechsel in eigenen Namespace mit eigenem Netzwerk und Prozess-IDs
 - ◇ `sudo unshare -n -p --fork --mount-proc sh`
- Testen ob Prozesse und Netzwerk isoliert sind:
 - ◇ `ps tree -p` # nur noch Prozesse in meinem Namespace sichtbar
 - ◇ `ping google.com` # kann nicht aufgelöst werden, weil Netzwerk fehlt
 - ◇ `ip addr` # nur loopback Netzwerkkarte vorhanden
 - ◇ `exit`





Übung: Linux Namespace wechseln (nsenter)

- Wechselt, via Desktop Icon, in die "dukmaster" VM
- Starten eines Containers, im Hintergrund, mittels Docker
 - ◇ `kubect1 run birdpedia --restart=Never \`
`--image=registry.gitlab.com/mc-b/birdpedia/birdpedia:1.0-alpine`
 - ◇ Anzeige alle Prozesse, es sollte ein neuer Prozess `birdpedia` vorhanden sein.
 - ◇ `ps tree -n -p -A -T`



- Wechsel in den Namespaces des Prozesses und damit in den Container

```
systemd(1) +- systemd-journal(326)
              +- containerd-shim(2991332) --- birdpedia(2991353)
```

- `sudo nsenter -t <pid> -a sh`

- Im Container, Testen ob Prozesse und Netzwerk isoliert sind:

- ◇ `ps tree -p` # zeigt nur Prozess in meinem Namespace
- ◇ `ping google.com` # kann aufgelöst werden, weil Docker ein Netzwerk Adapter installiert. Abbruch Ctrl+C
- ◇ `ip addr` # loopback und Docker Netzwerk Adapter vorhanden
- ◇ `ls -l; cat /etc/issue`
- ◇ `exit` # Container verlassen, zurück in VM
- ◇ `sudo kill -9 <pid>` # Prozess killen beendet auch den Container, Kontrollieren mit `ps tree -n -p -T -A`

